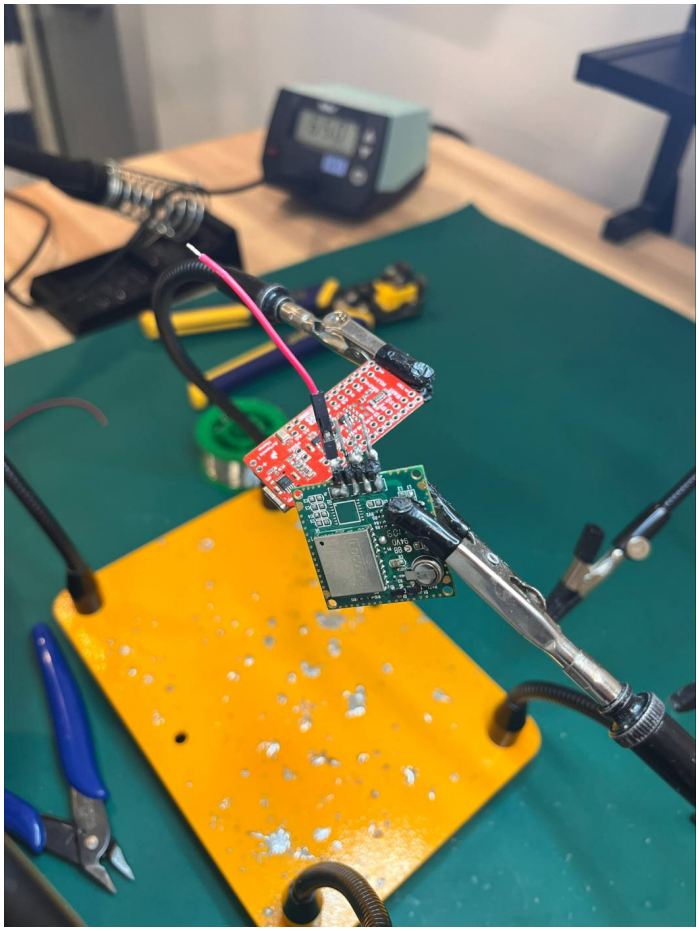
Cattle EID Tagging

Wearable tech that addresses an important and unique problem



Ryan Zucker, Ronald Nguyen, Janna Bailey, Olivia Homiller

12.05.2024

Meeting time: 4:35

Table of contents

0 Cover

1 Table of contents

2 Abstract

3 Reflection

4 Annotated Bibliography (MLA)

6 Decision Matrix

7 Expense sheet

8 Parts List

9 Time Management Sheet

10 Orthographic Drawings

11 Wiring Diagram

12 Photographs

14 Future work statement

15 Appendix

Abstract

Keeping cattle in free-range environments can be difficult due to obstacles like maintaining herds and minimizing disease damage. The USDA's Electronic Identification (EID) law now requires certain cattle and bison to have ear tags that can be read visually and electronically. Our device, an ordinary ear tag embedded with a GPS tracker and Arduino Nano, assigns data to the specific animal including location, name, age, and owner. Our digital, client-based approach will allow farmers to efficiently and effectively keep track of their herds. It also aims to improve disease control and study herd behavior through its low-power GPS tracking. Our device is capable of storing up to 24 hours of location data for such usage. In the future, we hope to be able to repackage this technology as a more durable and precise solution. (134 words)

Reflection

Our design process started out with many ideas that included many different methods and variables. We decided we wanted to choose something in agriculture and thought of designs including dogs. When we started brainstorming ideas for cattle, we thought of the already existing ear tags that cattle have. Once we started researching this topic and how we could add a new element to it, we found that the USDA's Electronic Identification (EID) law now requires certain cattle and bison to have ear tags that can be read visually and electronically. This would be the perfect time to create a new product that would follow the new guidelines. We wanted to add the aspect of GPS tracking. We mostly followed the engineering design process, with slowly increasing the functionality and percent of envisioned materials with each prototype. There were some ideas that we ended up not being able to do, like having a working QR code attached to a user interface that would have been able to have live information from the GPS. Overall, we had many ideas that we narrowed to our final prototype using research and understanding what we could complete given our time and resources. We went through different iterations of our prototypes before finalizing it and presenting it at the First Year Design Expo. (217 words)

Annotated Bibliography

Dior, Gior. “Sending Sensor Data via Bluetooth - SparkFun Learn.” *SparkFun*, 13 Nov. 2024, learn.sparkfun.com/tutorials/sending-sensor-data-via-bluetooth/all.

This is a direct explanation of how our device can transmit via Bluetooth. The next step of our design process was to display the data collected. This site provides detailed tutorials on how to get past seeing the data on a serial monitor and display the data in more helpful options such as OLED, Python graph, or a mobile device. This allows for less wiring outside of the device and is helpful when the cattle are in motion. This also aids in less snagging and making a more comfortable application for the cattle.

Koffman, Paul. “New EID Ear Tag Requirements for Cattle 2024 | Merck.” *Merck Animal Health USA*, 16 Oct. 2024, www.merck-animal-health-usa.com/species/cattle/cattle-insights/beef/new-cattle-eid-tag-requirements-2024

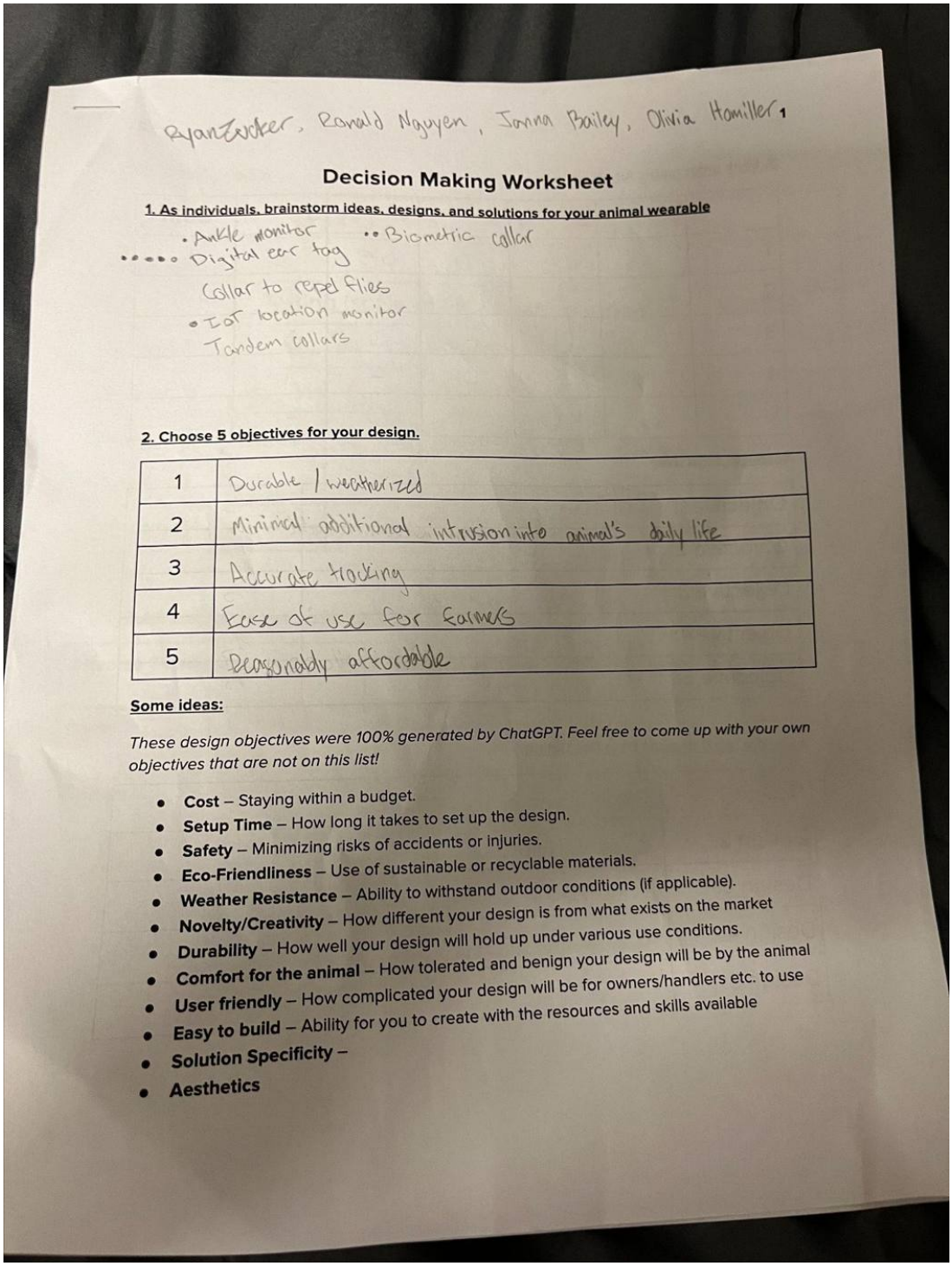
The new EID (Electronic Identification) law for cattle mandates that all livestock be tagged with a unique electronic ID to enhance traceability, improve biosecurity, and streamline disease control efforts. The law aims to bolster food safety and ensure more efficient management of animal health data. The tag we have aimed to create not only coincides with this new law, but also makes traceability and data collection easier than ever before. None of the new tags have

these modifications yet and thus in a real world scenario there would be a huge market for our product.

The Cattle Ranch. “Cattle Industry Controversy: Mandatory Electronic Identification (EID) Cattle Range.” *Cattlerange.com*, 2024, cattlerange.com/articles/2024/05/cattle-industry-controversy-mandatory-electronic-identification-eid/. Accessed 4 Dec. 2024.

This highlights the controversy surrounding the new EID law and electronification of the cattle tags. It mentions the proposal of using NAIS since 2005 and explains the pros and cons of this new identification method. This aided in producing our decision matrix and ultimately designing the tag as a finished product. By acknowledging what users and observers see as cons we did our best to steer clear and implement more of the pros for our final product.

Decision Matrix



Ryan Zecker, Ronald Nguyen, Jonna Bailey, Olivia Hamiller 1

Decision Making Worksheet

1. As individuals, brainstorm ideas, designs, and solutions for your animal wearable

- Ankle monitor
- Digital ear tag
- Biometric collar
- Collar to repel flies
- IoT location monitor
- Tandem collars

2. Choose 5 objectives for your design.

1	Durable / weatherized
2	Minimal additional intrusion into animal's daily life
3	Accurate tracking
4	Ease of use for farmers
5	Reasonably affordable

Some ideas:

These design objectives were 100% generated by ChatGPT. Feel free to come up with your own objectives that are not on this list!

- **Cost** – Staying within a budget.
- **Setup Time** – How long it takes to set up the design.
- **Safety** – Minimizing risks of accidents or injuries.
- **Eco-Friendliness** – Use of sustainable or recyclable materials.
- **Weather Resistance** – Ability to withstand outdoor conditions (if applicable).
- **Novelty/Creativity** – How different your design is from what exists on the market
- **Durability** – How well your design will hold up under various use conditions.
- **Comfort for the animal** – How tolerated and benign your design will be by the animal
- **User friendly** – How complicated your design will be for owners/handlers etc. to use
- **Easy to build** – Ability for you to create with the resources and skills available
- **Solution Specificity** –
- **Aesthetics**

3. Rank order matrix for objectives

		1	2	3	4	5	SUM
Durable	1	-	0.5	0	0.5	1	2
Minimal Intrusion	2	0.5	-	0	1	0	1.5
Accurate Tracking	3	1	1	-	1	0	3
Ease of Use	4	0.5	0	0	-	1	1.5
Affordability	5	0	0	1	0	-	1
		Durable	Minimal Intrusion	Accurate Tracking	Ease of Use	Affordability	

4. Weight objectives

Objective 1:	Durability	4/5
Objective 2:	Minimal Intrusion	3/5
Objective 3:	Accurate Tracking	5/5
Objective 4:	Ease of Use	3/5
Objective 5:	Affordability	2/5

5.List the top 5 designs from Dot Voting

Design 1:	Ear Tag
Design 2:	Biometric Collar
Design 3:	Ankle Monitor
Design 4:	IoT location monitor
Design 5:	

6. Decision matrix

Objectives:	1 Durability	2 Intrusion	3 Accuracy	4 Ease of use	5 Affordability	
Weights:	4	3	5	3	2	
	Rating (scale of 1-10) / Rating x Weight					TOTALS
Design 1	7/10 - 28	9/10 - 27	6/10 - 30	7/10 - 21	9/10 - 18	124 38/50
Design 2	9/10 - 36	6/10 - 18	10/10 - 50	7/10 - 21	5/10 - 10	134 37/50
Design 3	7/10 - 28	7/10 - 21	9/10 - 45	5/10 - 15	8/10 - 16	125 36/50
Design 4						
Design 5						

7. Adverse Consequences Matrix

Adverse Consequences	Probability to Occur	Seriousness Level	Threat Value
Scale	0 - no concern 10 - high concern	0 - no risk 10 - high risk	Probability x Seriousness
Design 1			
size / weight	2	4	8
Battery life	8	8	64
not watertight	2	8	16
Only wireless interfacing	4	8	32
		Total	120

Expense Sheet

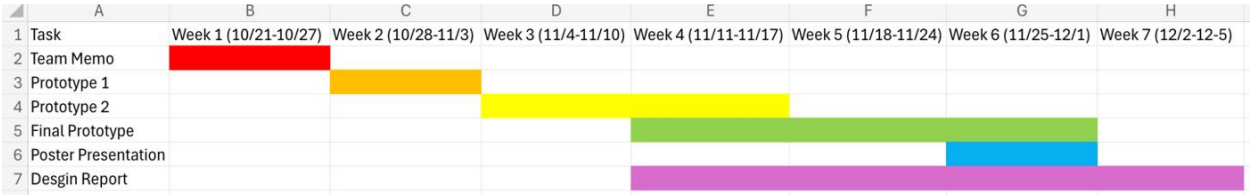
Tri-fold poster: $7

Sparkfun RedBoard Artemis Nano: $17.99

Parts List

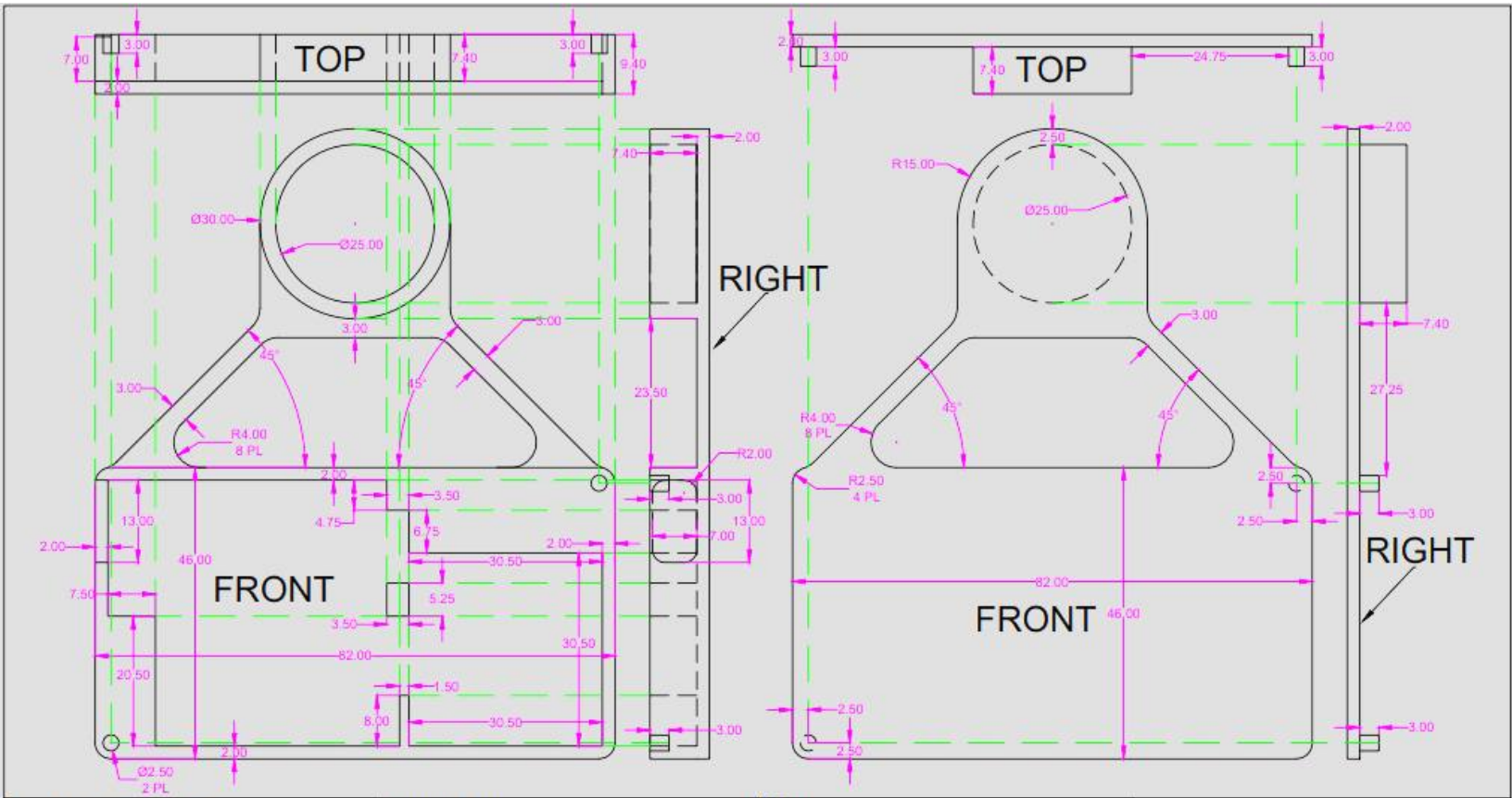
- Sparkfun RedBoard Artemis Nano
- 159GR-A C GPS Module
- Tri-fold Poster

Time Management Sheet



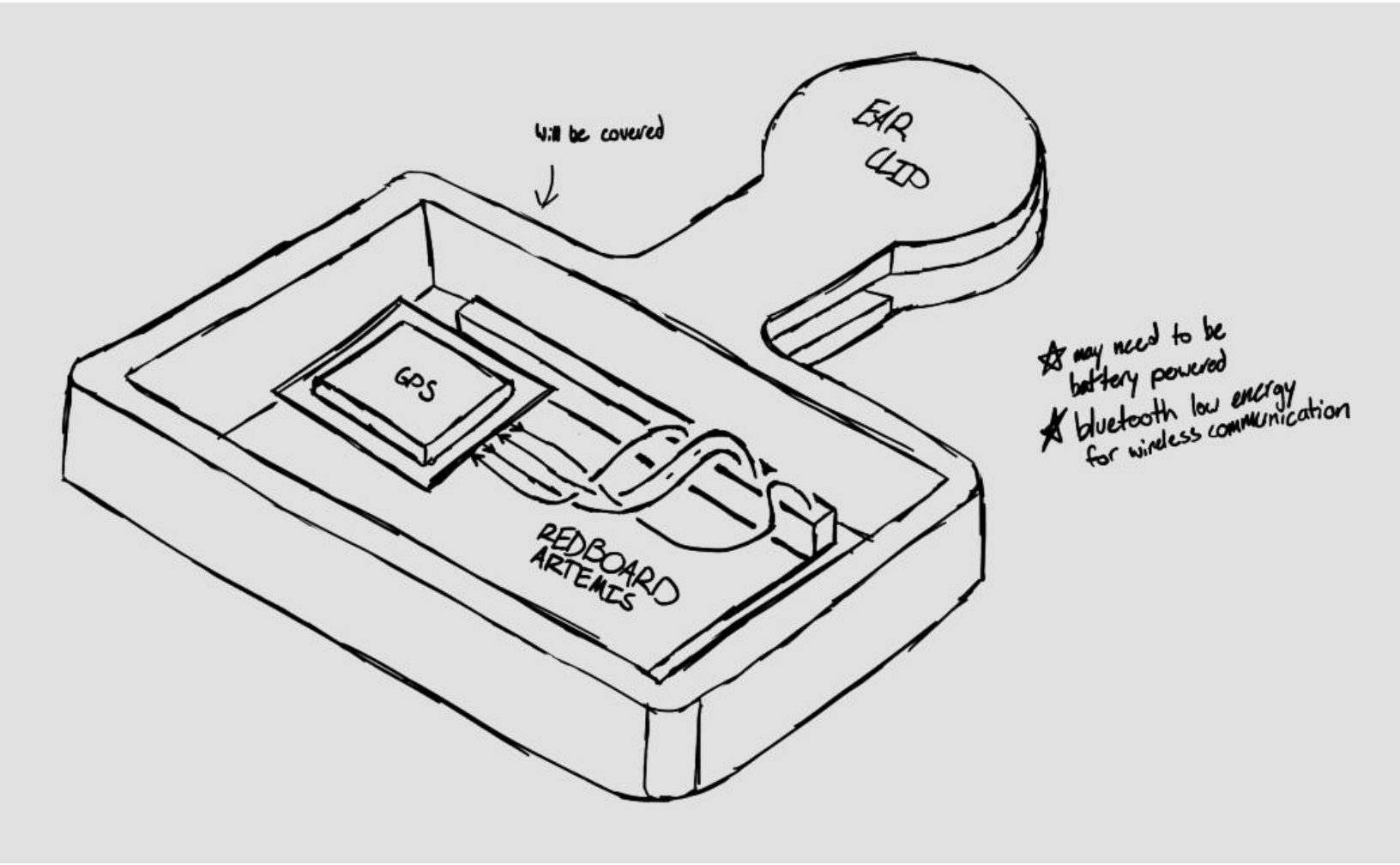
	A	B	C	D	E	F	G	H
1	Task	Week 1 (10/21-10/27)	Week 2 (10/28-11/3)	Week 3 (11/4-11/10)	Week 4 (11/11-11/17)	Week 5 (11/18-11/24)	Week 6 (11/25-12/1)	Week 7 (12/2-12-5)
2	Team Memo							
3	Prototype 1							
4	Prototype 2							
5	Final Prototype							
6	Poster Presentation							
7	Desgin Report							

Orthographic Drawings

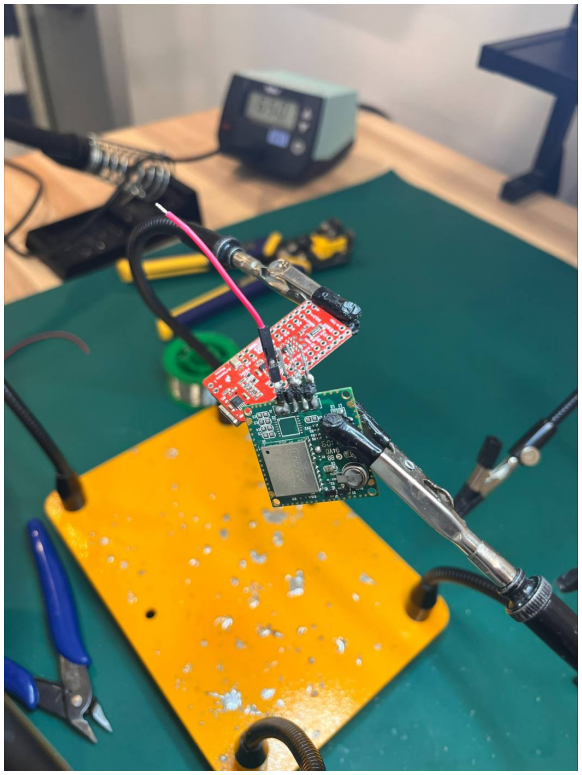


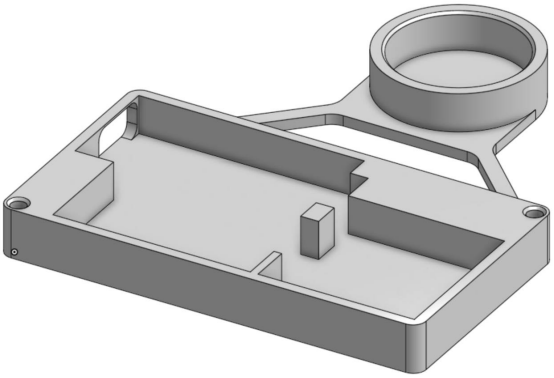
E-ID Ear Tag	Designed by: Janna Bailey, Olivia Homiller, Ronald Nguyen, Ryan Zucker	Units: Millimeters
An orthographic model of the case of an E-ID ear tag. Left and right models are designed to interlock and enclose circuitry.	Production Date: 12 / 02 / 2024	Scale: 1 in paper ≈ 29.5 mm real
	Professor: Cantilina (GE1501-12177)	

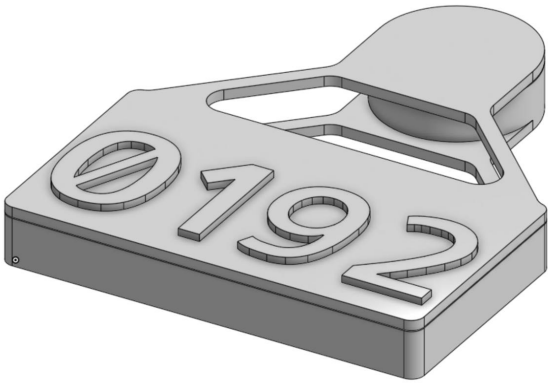
Concept Drawing:

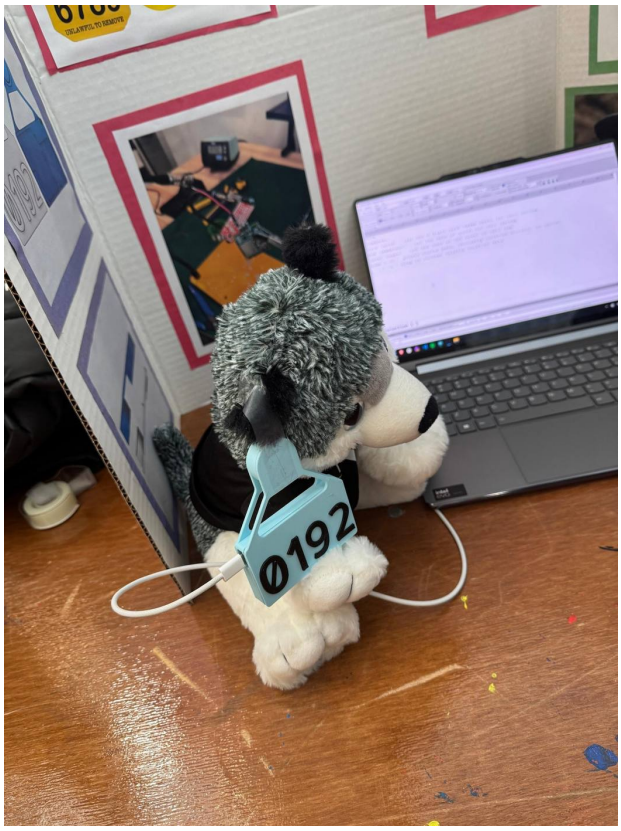


Photographs

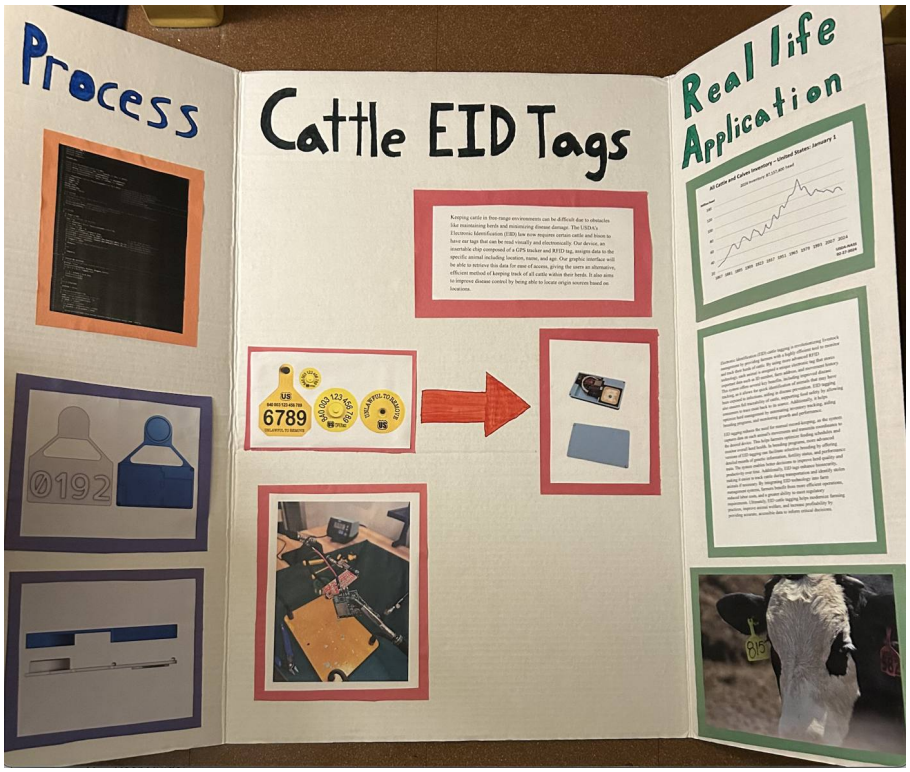







0192




Process
Cattle EID Tags
Real life Application
Keeping cattle in free-range environments can be difficult due to obstacles like maintaining herds and minimizing disease damage. The USDA's Electronic Identification (EID) law now requires certain cattle and bison to have ear tags that can be read visually and electronically. Our device, an insertable chip composed of a GPS tracker and RFID tag, assigns data to the specific animal including location, name, and age. Our graphic interface will be able to retrieve this data for ease of access, giving the users an alternative, efficient method of keeping track of all cattle within their herds. It also aims to improve disease control by being able to locate origin sources based on locations.
6789
0192

Future Work Statement

If given more time and funds, the precision and accuracy of our device would improve significantly. Our decision matrix was impacted most by cost and overall efficiency of usage. Because of this, we decided upon using a Sparkfun Artemis Arduino nano and GPS component from FYELIC. The cost restraint of $40 dollars made the Artemis our best option cost wise to get our desired effect. The usage and construction of a metal or nylon could have been an option with more funds and time. Nylon is generally more expensive than polyester due to its durability and strength. Nylon Rope or a sheet, which could have been utilized as a protectant for our device, is far too expensive and difficult to obtain in small amounts. Due to this, we agreed upon 3D printing our case for the EID tagging device. The plastic coating imitates what could be achieved with high grade nylon products. Additionally, the GPS (66 channel) from FYELIC could have also been replaced with the GPS/GNSS Embedded Antenna - 1m (SMA) from the Spark fun website. The embedded Antenna would provide reception of Beidou satellites where other more basic GPS antennas can't. (193 words)

.

Appendix

Eartag Arduino Code:

```
#include <TinyGPS.h>
#include <EEPROM.h>
#include <time.h>
#include <stdlib.h>

TinyGPS gps;

static void smartDelay(unsigned long ms);
static float saveFloat(float val, float invalid, int len, int prec);
static String readFromEEPROM(int startAddress);
static void writeToEEPROM(const String &string, int startAddress);
static void dumpDataToPC(float **arr, int *sIndex, int *logged);

float **positions = (float **)malloc(sizeof(float *) * 2881);  // memory allocation for coordinate array
int sIndex = 0;                                                // starting index of current block of locations
int logged = 0;                                                // length of current block of locations
int reps = 60;                                                 // variable to make sure serial responses are fast but gps readings stay slower
String scmd;                                                   // string to store command
String scmd_arg;                                               // string to store command args
bool printed = false;                                          // bool to ensure we don't repeatedly output data after it's already been sent

void setup() {  // initialize EEPROM / start Serial + Serial1

  EEPROM.init();
  // for (int i = 0; i < EEPROM.length(); i++) EEPROM.write(i, 0); // function to clear EEPROM if needed
  Serial.begin(9600);    // for communicating with connected PC
  Serial1.begin(38400);  // for getting GPS data
}

void loop() {
  if (reps >= 2) {  // 30s has passed, log location
    reps = 0;
    float *position = (float *)malloc(sizeof(float) * 2);
    float flat, flon;
    unsigned long age = 0;
    int cIndex = (logged + sIndex) % 2880;                              // current index in memory
    gps.f_get_position(&flat, &flon, &age);                             // get location data from GPS module
    position[0] = saveFloat(flat, TinyGPS::GPS_INVALID_F_ANGLE, 10, 6);  // save latitude to memory
    position[1] = saveFloat(flon, TinyGPS::GPS_INVALID_F_ANGLE, 10, 6);  // save longitude to memory
    positions[cIndex] = position;                                       // save to massive malloc array
    if (logged >= 2880) {                                               // 24+hrs passed, begin overwriting old data
      sIndex++;
    } else {
      logged++;
    }
  }
  if (Serial && !printed) {  // automatically run dump command when serial port connects
    printed = true;
    dumpDataToPC(positions, &sIndex, &logged, false);
    for (int i = 0; i < logged; i++) {
      int j = (sIndex + i) % 2880;
      free(positions[i][0]);
      free(positions[i][1]);
    }
  } else if (!Serial) {
    printed = false;
```

```
  }
  if (Serial.available() > 0) {  // when command received from serial port
    scmd = Serial.readStringUntil(' ');
    scmd_arg = Serial.readString();
    if (scmd == "uuid") {  // save UUID to EEPROM
      int uuidInt = atoi(scmd_arg.c_str());
      if (uuidInt < 0 || uuidInt > 9999) {
        Serial.println("UUID value outside accepted range! Acceptable values: 0000 - 9999");
      } else {
        Serial.println("Saving UUID...");
        writeToEEPROM(scmd_arg, 0);
        Serial.println("UUID saved!");
      }
    } else if (scmd == "dob") {  // save DOB to EEPROM
      const char *dob_c_str = scmd_arg.c_str();
      int mo = (dob_c_str[0] - '0') * 10 + (dob_c_str[1] - '0');  // parse inputted dob into its parts
      int da = (dob_c_str[2] - '0') * 10 + (dob_c_str[3] - '0');  // why do i have to cast char* to each of these????
      int yr = (dob_c_str[4] - '0') * 1000 + (dob_c_str[5] - '0') * 100 + (dob_c_str[6] - '0') * 10 + (dob_c_str[7] - '0');
      if ((mo < 1 || mo > 12) || (da < 1 || da > 31) || (yr < 1970)) {  // check that it fits unix time standard
        Serial.println("Invalid date!");
      } else {
        Serial.println("Saving date of birth...");
        struct tm hr_time;            // human-readable time struct to convert to unix;
        hr_time.tm_year = yr - 1900;  // year
        hr_time.tm_mon = mo - 1;      // month
        hr_time.tm_mday = da;         // day
        hr_time.tm_hour = 0;
        hr_time.tm_min = 0;
        hr_time.tm_sec = 0;

        time_t unix = mktime(&hr_time);       // convert from human-readable time to unix
        writeToEEPROM(String((int)unix), 5);  // store in EEPROM
        Serial.println("Date of birth saved!");
      }
    } else if (scmd == "owner") {  // save OWNER to EEPROM
      Serial.println("Saving owner...");
      writeToEEPROM(scmd_arg, 18);
      Serial.println("Owner saved!");
    } else if (scmd == "dump") {  // print all saved data to serial (UUID, DOB, owner, location data);
      if (scmd_arg == "-i") {
        dumpDataToPC(positions, &sIndex, &logged, true);
      } else {
        dumpDataToPC(positions, &sIndex, &logged, false);
      }
      for (int i = 0; i < logged; i++) {  // free old position data (yes i know this fragments the hell out of the stack but idc
        int j = (sIndex + i) % 2880;
        free(positions[i][0]);
        free(positions[i][1]);
      }
    } else if (scmd == "help") {
      delay(1);
      Serial.println("Commands:");
      delay(1);
      Serial.println("uuid <uuid> - set the 4-digit uuid (0000-9999) for this eartag");
      delay(1);
      Serial.println("dob <MMDDYYYY> - set the date of birth for this eartag");
```

1501sp-eartag-onboard.ino

```
      delay(1);
      Serial.println("owner <name> - set the name of the owner of this tag");
      delay(1);
      Serial.println("dump <-i> - prints stored data, including location history, to serial");
      delay(1);
      Serial.println("\t  -> -i - flag to include invalid location data");
    } else {
    }
  }
  reps++;
  smartDelay(500);  // wait 500ms between loops
}

static void smartDelay(unsigned long ms) {  // function adapted from Mikel Hart's TinyGPS library
  unsigned long start = millis();
  do {
    while (Serial1.available())
      gps.encode(Serial1.read());
  } while (millis() - start < ms);
}

static float saveFloat(float val, float invalid, int len, int prec) {  // function adapted from Mikel Hart's TinyGPS library
  float n;
  if (val == invalid) {
    n = NULL;
  } else {
    n = val;
  }
  smartDelay(0);
  return n;
}

static String readFromEEPROM(int startAddress) {  // function to read strings from EEPROM
  int stringLen = EEPROM.read(startAddress);
  char data[stringLen + 1];
  for (int i = 0; i < stringLen; i++) {
    data[i] = EEPROM.read(startAddress + i + 1);
  }
  data[stringLen] = '\0';
  return String(data);
}

static void writeToEEPROM(const String &string, int startAddress) {  // function to write strings to EEPROM
  byte len = string.length();
  EEPROM.write(startAddress, len);
  for (int i = 0; i < string.length(); i++) {
    EEPROM.write(startAddress + i + 1, string[i]);
  }
  return;
}

static void dumpDataToPC(float **arr, int *sIndex, int *logged, bool include) {  // prints UUID, DOB, owner, and location data over serial
  Serial.println();
  delay(1);
  Serial.print("Data for ID ");
  delay(1);
```

```
  delay(1);
  Serial.print(readFromEEPROM(0));  // get id from EEPROM
  delay(1);
  Serial.println(":");
  struct tm hr_time;  // convert EEPROM-stored unix time to human-readable time
  char buf[80];
  time_t unix = (time_t)readFromEEPROM(5).toInt();
  hr_time = *localtime(&unix);
  strftime(buf, sizeof(buf), "%m / %d / %Y", &hr_time);
  delay(1);
  Serial.print("- Date of Birth: ");
  delay(1);
  Serial.println(String(buf));
  delay(1);
  Serial.print("- Owner: ");
  delay(1);
  Serial.println(readFromEEPROM(18));
  delay(1);
  Serial.println("- Location Data (past 24h):");
  int t_cIndex = 0;                   // temp cIndex
  int t_sIndex = *sIndex % 2880;      // temp sIndex
  int t_logged = min(*logged, 2880);  // temp logged
  for (int i = 0; i < t_logged; i++)  // loop through the coordinate array, printing pairs to serial and over bluetooth
  {
    t_cIndex = (t_sIndex + i) % 2880;
    if ((arr[t_cIndex][0] != NULL && arr[t_cIndex][1] != NULL) || include) {  // exclude invalid data pairs unless user explicitly includes -i flag
      delay(1);
      Serial.print('[');
      delay(1);
      Serial.print(arr[t_cIndex][0], 6);
      delay(1);
      Serial.print(", ");
      delay(1);
      Serial.print(arr[t_cIndex][1], 6);
      delay(1);
      Serial.println(']');
    }
  }
  delay(1);
  Serial.println("Data dump complete!");
  *logged = 0;
  *sIndex = 0;
}

```

C++ User Guide:

```cpp
#include <iostream>

using namespace std;

int main() {
    char input; // dummy variable
    cout << "Welcome to the E-ID Eartag setup! Press ENTER to continue at each step...";
    cin.get(input); // wait for user to press the enter key
    cout << "Step 1) Please get your cow.";
    cin.get(input);
    cout << "Step 2) Ensure you can attach the eartag to it. Remove previous eartag if applicable.";
    cin.get(input);
    cout << "Step 3) Clean the spot where you will attach the eartag, the eartag applicators, and the eartag thoroughly.";
    cin.get(input);
    cout << "Step 4) Mount the eartag onto the eartag applicators.";
    cin.get(input);
    cout << "Step 5) Clamp the applicators on the spot you wish to apply the eartag.";
    cin.get(input);
    cout << "Step 6) Release the applicators and once again clean the spot around where the eartag is now attached.";
    cin.get(input);
    cout << "Step 7) Connect the eartag via USB to this laptop. The port is on the side of the eartag.";
    cin.get(input);
    cout << "Step 8) Open a serial terminal using an app such as HTerm at 9600 baud.";
    cin.get(input);
    cout << "Step 9) Send the command 'dob <MMDDYYYY> to set the date of birth of the cow.";
    cin.get(input);
    cout << "Step 10) Send the command 'owner <name>' to set the owner of the cow.";
    cin.get(input);
    cout << "Step 11) Send the command 'dump' to output stored data including ID, owner, and date of birth to check successful storage.";
    cout << "\n         If the ID does not match that of what is on the case, run the command 'uuid <id>' to set it to match.";
    cout << "\n         If data input with the 'dob' or 'owner' commands do not match, re-run them or unplug and replug the device.";
    cin.get(input);
    cout << "NOTE: Every time the device is connected to a serial terminal, it will output a log of coordinates the cow has visited in the past 24 hours.";
    cout << "\n      Store this data however you would like. This data can be manually retrieved using the 'dump' command.";
    cout << "\n      All commands can be viewed by sending the 'help' command via serial terminal.";
    cin.get(input);
    cout << "Thank you for using the E-ID Eartag setup program. Press ENTER to exit.";
    cin.get(input);
}
```